

An **API (Application Programming Interface)** is a mechanism that allows different software systems to communicate with each other.

A simple way to explain it to students is:

An API is like a waiter in a restaurant.

You (the customer) make a request → the waiter takes the request to the kitchen → the kitchen processes it → the waiter brings the result back.

In digital business, APIs allow companies to **use, share, combine, and commercialise digital capabilities** without having to build everything themselves.

1. API as a Business Model

What does it mean?

Traditionally, a company might build a complete software application and sell it directly to customers.

With an **API-based business model**, a company exposes some of its capabilities through APIs so that other companies can integrate them into their own applications.

For example:

A company does not need to build its own payment system.
It can integrate a payment API provided by a payment company.

Simple example: Online shopping

Imagine a student creates an online clothing store.

The store needs:

1. Product catalogue
2. Shopping cart
3. Payment
4. Delivery
5. Email notifications
6. Maps/location
7. Customer analytics

The student could build everything from scratch—but that would take a lot of time.

Instead, the application can use APIs:

Online Store

- Payment API
- Delivery API
- Maps API
- Email API
- Analytics API

The APIs become the **building blocks** of the digital business.

Real-world example: Stripe

stripe API

Q Find anything

Balance

Balance Transactions

Charges

The charge object

Create a charge

Retrieve a charge

Update a charge

Capture a charge

List all charges

Customers

Disputes

Events

Files

File Links

Mandates

PaymentIntents

SetupIntents

SetupAttempts

Payouts

Refunds

Tokens

PAYMENT METHODS

PaymentMethods

Bank Accounts

Cards

Sign In

Create a charge

To charge a credit card or other payment source, you create a `Charge` object. If your API key is in test mode, the supplied payment source (e.g., card) won't actually be charged, although everything else will occur as if in live mode. (Stripe assumes that the charge would have completed successfully).

Parameters

amount REQUIRED	Amount intended to be collected by this payment. A positive integer representing how much to charge in the smallest currency unit (e.g., 100 cents to charge \$1.00 or 100 to charge ¥100, a zero-decimal currency). The minimum amount is \$0.50 US or equivalent in charge currency. The amount value supports up to eight digits (e.g., a value of 99999999 for a USD charge of 999,999.99).
currency REQUIRED	Three-letter ISO currency code, in lowercase. Must be a supported currency.
customer optional	The ID of an existing customer that will be charged in this request.
description optional	An arbitrary string which you can attach to a <code>Charge</code> object. It is displayed when in the web interface alongside the charge. Note that if you use Stripe to send automatic email receipts to your customers, your receipt emails will include the <code>description</code> of the charge(s) that they are describing.
metadata optional dictionary	Set of key-value pairs that you can attach to an object. This can be useful for storing additional information about the object in a structured format. Individual keys can be unset by posting an empty value to them. All keys can be unset by posting an empty value to <code>metadata</code> .

POST /v1/charges

Select library

```
$ # 'source' is obtained with Stripe.js; see https://stripe.com/docs/payments
curl https://api.stripe.com/v1/charges \
  -u sk_test_4eC39HqLyJWDarjt1Lzdp7dc: \
  -d amount=2000 \
  -d currency=usd \
  -d source=tok_mastercard \
  -d description="My First Test Charge (created for API docs)"
```

RESPONSE

```
{
  "id": "ch_3HV7M62eZvKYlo2C18Z7Qe06",
  "object": "charge",
  "amount": 2000,
  "amount_captured": 0,
  "amount_refunded": 0,
  "application": null,
  "application_fee": null,
  "application_fee_amount": null,
  "balance_transaction": "txn_1032HU2eZvKYlo2CEPtcnUv1",
  "billing_details": {
    "address": {
      "city": null,
      "country": null,
      "line1": null,
      "line2": null,
      "postal_code": null,
      "state": null
    },
    "email": null,
    "name": "Jenny Rosen",
    "phone": null
  },
  "captured": true,
  "created": 1490115123,
  "currency": "usd",
  "description": "My First Test Charge (created for API docs)",
  "disputed": false,
  "failure_code": null,
  "failure_message": null,
  "invoice": null,
  "livemode": false,
  "metadata": {},
  "on_behalf_of": null,
  "order": null,
  "paid": true,
  "receipt_email": null,
  "review": null,
  "shipping": null,
  "source": "tok_mastercard",
  "status": "succeeded",
  "transfer_data": null,
  "transfer_method": null
}
```


For example:

Customer

↓

Online shopping website

↓

Stripe API

↓

Payment processing

Instead of the online store developing an entire payment infrastructure itself, it can integrate payment functionality through Stripe.

Business value

The API allows the business to:

- Accept payments
- Reduce development effort
- Integrate payment functionality into its application
- Build a smoother checkout experience

Key lesson

API as a business model = "We make our digital capability available for other businesses to use."

2. API Partner Ecosystem Networks

This concept goes one step further.

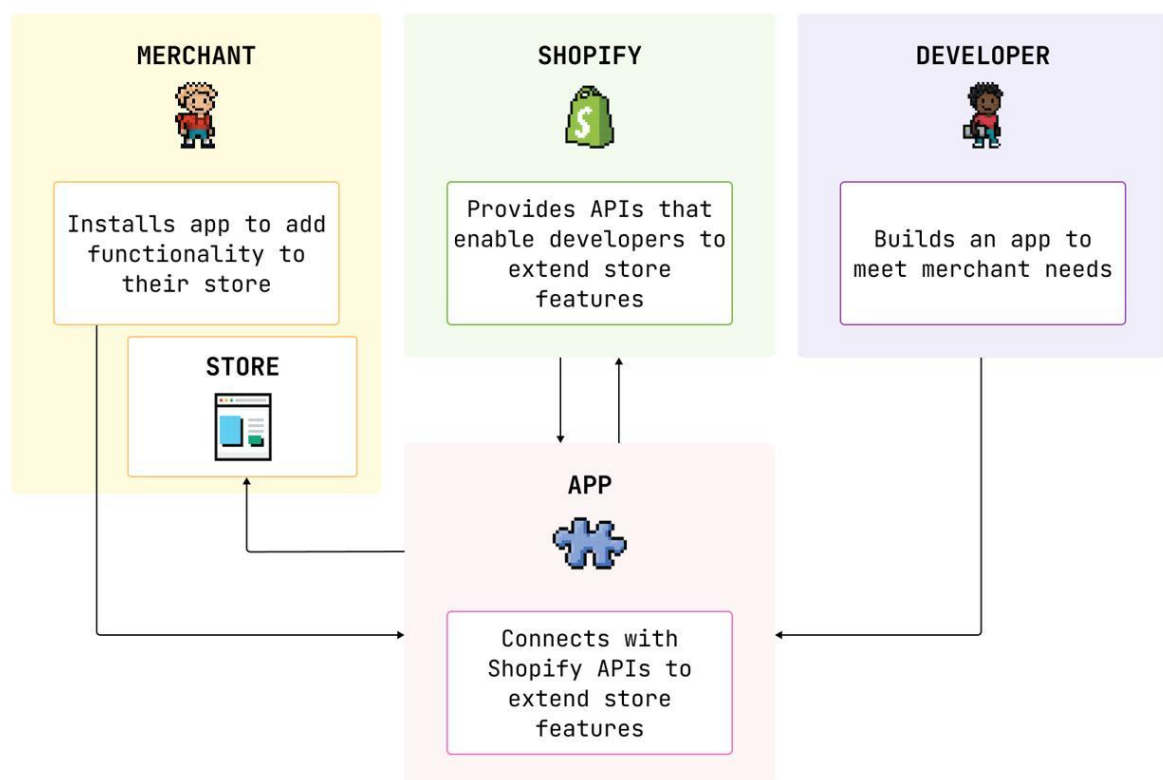
A company doesn't just provide an API to one customer. It can create an **ecosystem of many developers, businesses, and technology partners**.

Think of it as a digital LEGO system.

The platform provides the basic LEGO blocks.

Other companies use those blocks to create new products and services.

Example: Shopify



Hello martytest3

Get the very best apps for your store

Popular with stores like yours

Shopify Email 4.5 ★ (1,826) • Free to install Email marketing made for commerce	Facebook & Instagram 4.2 ★ (6,790) • Free to install The official app from Meta. Sell on Facebook and Instagram.	Google & YouTube 4.6 ★ (8,685) • Free to install Access the best of Google and YouTube
Shopify Inbox 4.8 ★ (5,450) • Free Connect with shoppers and drive sales with chat	Shopify Search & Discovery 4.6 ★ (1,105) • Free Customize your storefront search and discovery experience	TikTok 4.8 ★ (23,470) • Free to install Showcase your products to TikTok's users and increase sales
Pinterest 4.6 ★ (5,919) • Free to install Put your products where people go to find ideas to try and buy	Klaviyo: Email Marketing & SMS 4.6 ★ (2,753) • Free plan available Power smarter digital relationships with email marketing & SMS	Instafeed - Instagram Feed 4.9 ★ (1,712) • Free plan available Boost sales with Instagram gallery, shoppable video & Reels. Built for Shopify
Printful: Print on Demand 4.6 ★ (6,974) • Free plan available Sell custom print and embroidery items with no upfront costs	PageFly Landing Page Builder 4.9 ★ (11,187) • Free plan available Landing page builder, home, product page, store custom design	Zendrop - Dropshipping & POD 4.8 ★ (9,719) • Free plan available Start drop shipping in 30 sec, 1M+ products. Fast delivery

We Support the Most Popular Apps and Integrations

Marketing	Recommendations	Customer Support	Search	Reviews	Loyalty
Returns	Wishlist	Shipping			

Shopify provides an e-commerce platform that can be extended through apps and integrations.

A Shopify merchant might use:

Shopify

- Accounting app
- Email marketing app

- Inventory system
- Shipping service
- Customer relationship management
- Analytics tool
- Payment service

These third-party applications can connect to Shopify through APIs and other integration mechanisms.

Example scenario

Imagine **ABC Fashion Store** uses Shopify.

The business wants:

- Shopify → online store
- Xero → accounting
- Mailchimp → email marketing
- Shipping provider → delivery
- Facebook/Instagram → social commerce
- Analytics → customer insights

Instead of developing every function itself, ABC Fashion Store connects different systems.

Ecosystem effect

Shopify benefits because:

More developers → more apps → more capabilities → more attractive platform → more merchants

Developers benefit because:

Shopify users → potential customers for their applications

Merchants benefit because:

More integrations → more choices and functionality

This is an **API partner ecosystem**.

API Ecosystem vs. Traditional Software

A useful comparison for students:

Traditional Approach	API Ecosystem Approach
Company builds everything itself	Company integrates external capabilities
Large development effort	Reuses existing services
Fewer integrations	Many integrations
Slower innovation	Faster experimentation
Closed system	More open ecosystem
Difficult to scale features	New partners can add capabilities

Example

A restaurant developing a food-delivery application could build:

Traditional approach

Own payment system + own maps + own messaging + own analytics + own delivery system

API approach:

Payment API + Google Maps API + messaging API + analytics API + delivery API

The second approach can significantly reduce the amount of functionality the restaurant needs to develop

3. Cross-Industry Co-Creation

What does it mean?

Cross-industry co-creation happens when companies from different industries combine their capabilities to create a new customer experience or business solution.

For example:

Banking + Retail + Technology

can create new digital financial services.

Or:

Healthcare + Technology + Telecommunications

can create remote healthcare services.

The important idea is:

One company does not have to possess every capability required to create the final customer value.

Example: Banking + Retail

Imagine a customer buys a laptop from an online electronics retailer.

The retailer could integrate a financial services API.

The process might be:

Customer

↓

Online Retailer

↓

Bank/FinTech API

↓

Credit/Payment service

↓

Purchase completed

The retailer provides the product.

The financial institution provides financial/payment capabilities.

The technology platform connects them.

Value created

Retailer:

- Easier checkout
- Additional payment options
- Potentially higher conversion

Bank/FinTech:

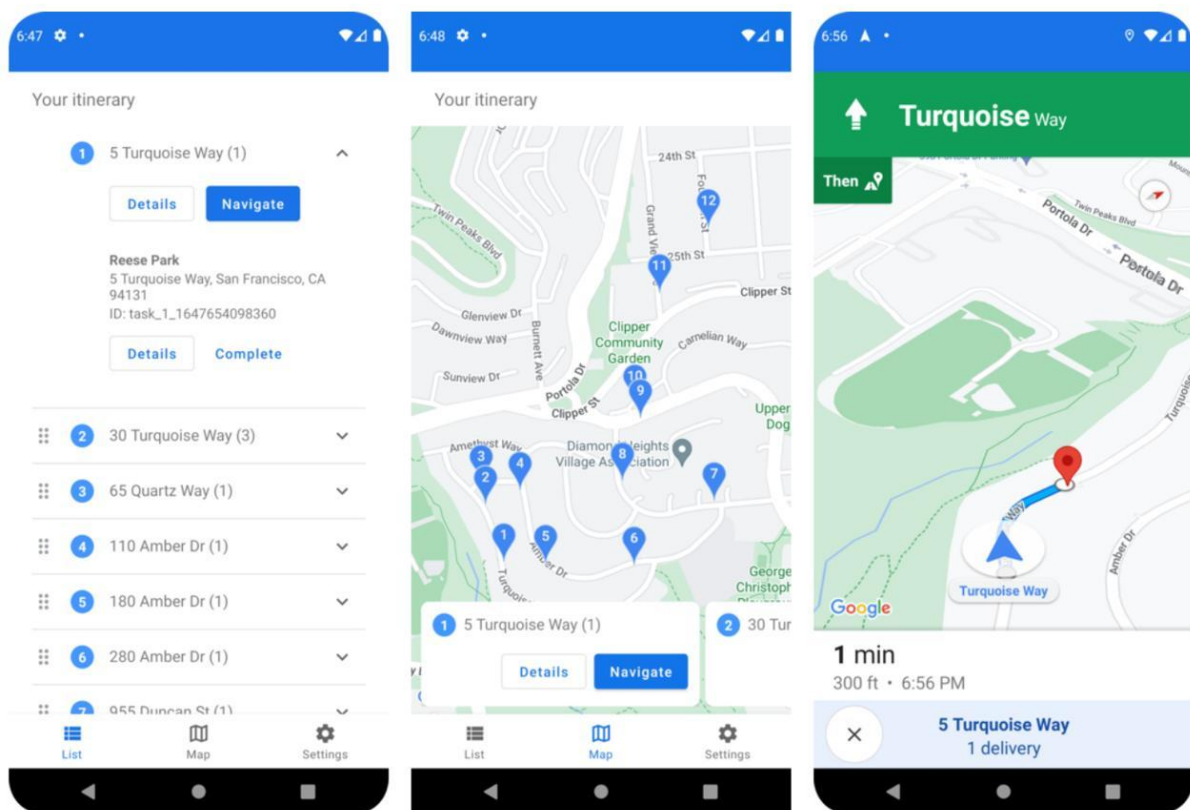
- Access to potential customers

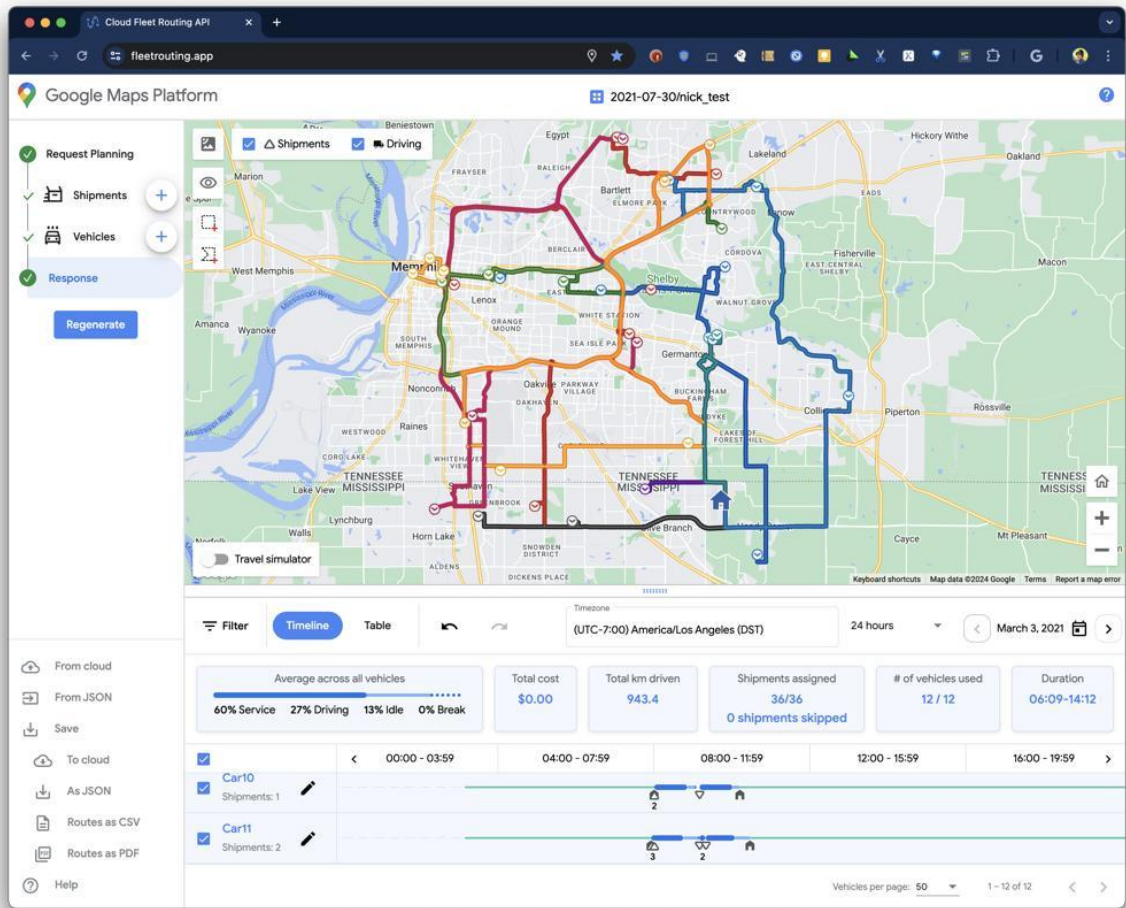
Customer:

- Convenient payment/financing experience

This is cross-industry value creation.

Example: Google Maps APIs + Other Industries

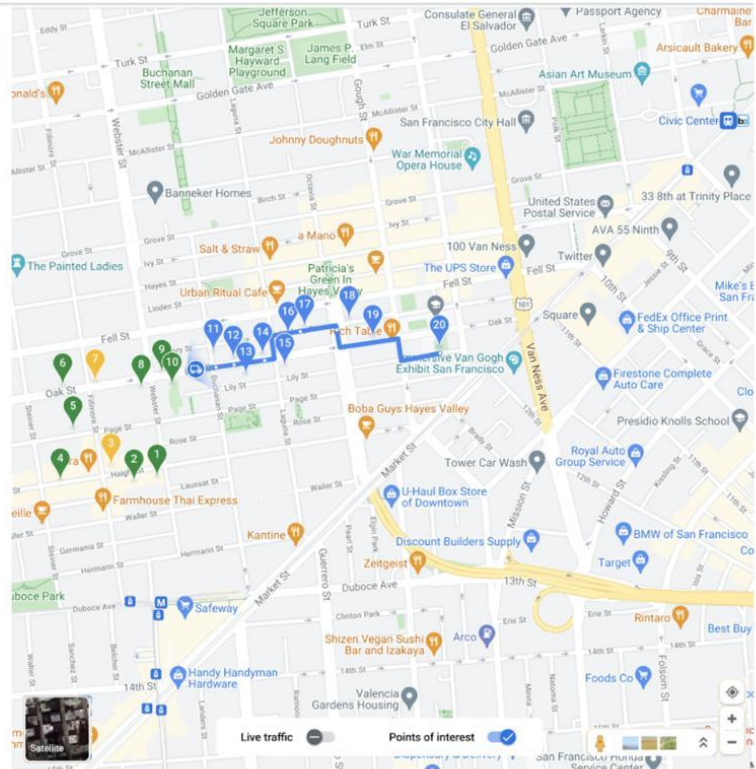




Search by vehicle ID or shipment ID

VEHICLE
315529

#	Shipment ID	Type	Status	Time
1	397976787	Drop off	SUCCESS	11:32
2	397825768	Drop off	SUCCESS	11:34
3	397823456	Drop off	ATTEMPTED	11:41
4	397825674	Pick up	SUCCESS	11:42
5	39782234	Pick up	SUCCESS	11:45
6	397823529	Drop off	SUCCESS	12:36
7	397823756	Drop off	ATTEMPTED	12:56
8	397823456	Drop off	SUCCESS	1:07
9	397834563	Drop off	SUCCESS	1:10
10	397834563	Drop off	SUCCESS	1:17
11	397834563	Drop off	UNATTEMPTED	-
12	397834563	Pick up	UNATTEMPTED	-
12	397834563	Drop off	UNATTEMPTED	-
13	397834563	Drop off	UNATTEMPTED	-
14	397834563	Drop off	UNATTEMPTED	-



Google provides mapping and location technologies that developers can integrate into applications.

Consider a **food delivery application**.

The food delivery company does not need to create its own global mapping infrastructure.

It can integrate mapping/location functionality.

Food delivery process

Customer enters address



Map/location API identifies location



Restaurant location is displayed



Delivery route is calculated



Driver navigates to customer

↓

Customer tracks delivery

A mapping capability originally developed as a technology service can therefore become part of another industry's customer experience.

Cross-industry connection

Technology

-

Restaurant

-

Logistics

=

Food delivery ecosystem

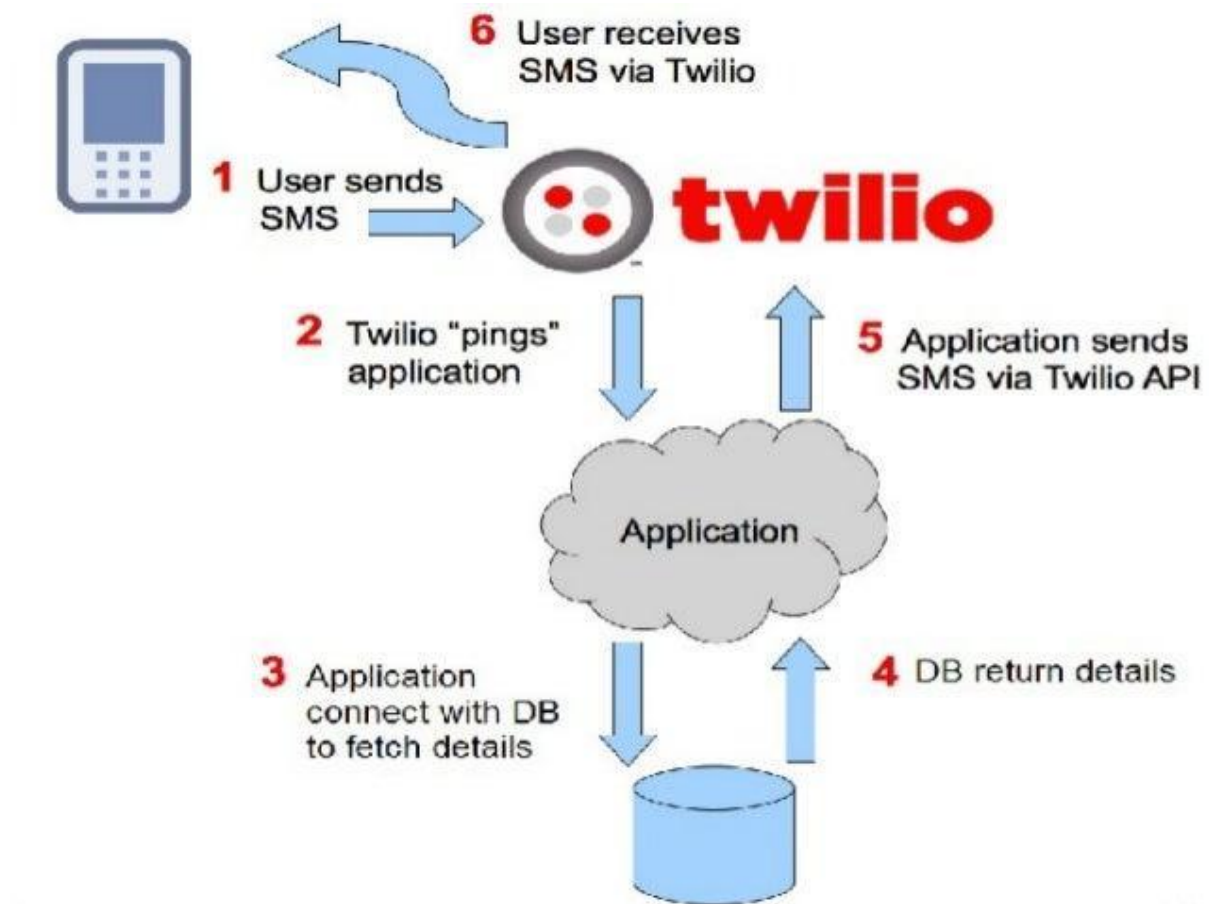
Communication & Support APIs

APIs can also provide communication capabilities.

Examples include:

- SMS
- WhatsApp messaging
- Email
- Video
- Voice calls
- Customer support

Example: Twilio



Twilio Docs SMS Voice Runtime Video Studio All docs... English SDKs Help Search Log in Sign up

Programmable SMS

- Getting Started
 - How to Work with your Free Twilio Trial Account
 - PHP Quickstart
 - .NET Framework Quickstart
 - .NET Core Quickstart
 - Java Quickstart
 - Node.js Quickstart
 - Ruby Quickstart
 - Python Quickstart
- API Reference
 - API Overview
 - Message Resource
 - MessageFeedback Resource
 - Media Resource
 - Service Resource
 - Pricing
- TwIML
 - Overview
 - <Message>
 - <Redirect>
- Tutorials
 - How to receive and reply to SMS messages

Send SMS with Twilio Programmable Messaging

Twilio makes sending and receiving SMS easy. Find the documentation, sample code, and developer tools you need to build exactly what you want, fast. We'll handle the complexity of mobile carrier and global regulations. Let's get building.

[Send your first message >](#)

TAKE THE NEXT STEP WITH TWILIO SMS

[GET STARTED](#) [BUILD YOUR APP](#) [ADVANCED FEATURES](#)

Rate this page: ★★★★★

1

2

```
const MessagingResponse =
  require("twilio").twiml.MessagingResponse;

app.post("/sms", (req, res) => {
  const twiml = new MessagingResponse();
  twiml.message("Thanks for signing up!");
  res.end(twiml.toString());
});
```

View complete examples: 71

3

Thanks for signing up!

Send an SMS with Twilio's API

To send a new *outgoing* message from a Twilio phone number to an outside number, make an HTTP POST to your account's [Message resource](#):

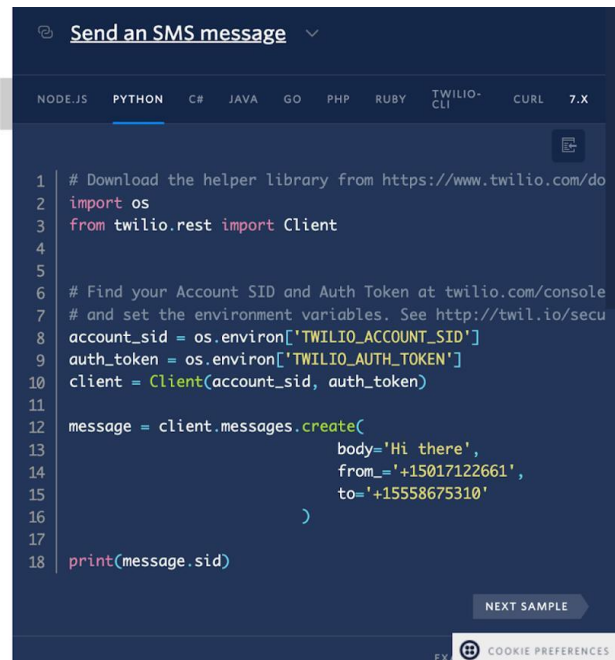
```
1 | /2010-04-01/Accounts/{AccountSid}/Messages
```

You can post directly to the API with [cURL](#) or use one of our six supported helper libraries to send messages with [C#](#), [Java](#), [Node.js](#), [PHP](#), [Python](#), [Ruby](#), or [Go](#).

When creating a new message via the API, include the parameters [To](#), [From](#), and [Body](#).

To

This parameter determines the destination phone number for your SMS message. Format this number with a '+' and a country code, e.g., `+16175551212` ([E.164 format](#)).



The screenshot shows the Twilio console interface for sending an SMS message. At the top, there's a tab bar with options: NODE.JS, PYTHON (selected), C#, JAVA, GO, PHP, RUBY, TWILIO-CLI, CURL, and 7.X. Below the tabs is a code editor with a Python script. The script imports the Twilio client, sets environment variables for the account SID and auth token, and then creates a message with a body of 'Hi there', a from number of '+15017122661', and a to number of '+15558675310'. The script prints the message SID. At the bottom right, there's a 'NEXT SAMPLE' button and a 'COOKIE PREFERENCES' link.

```
1 | # Download the helper library from https://www.twilio.com/docs
2 | import os
3 | from twilio.rest import Client
4 |
5 |
6 | # Find your Account SID and Auth Token at twilio.com/console
7 | # and set the environment variables. See http://twil.io/secu
8 | account_sid = os.environ['TWILIO_ACCOUNT_SID']
9 | auth_token = os.environ['TWILIO_AUTH_TOKEN']
10 | client = Client(account_sid, auth_token)
11 |
12 | message = client.messages.create(
13 |     body='Hi there',
14 |     from_='+15017122661',
15 |     to='+15558675310'
16 | )
17 |
18 | print(message.sid)
```

Twilio provides communication APIs that developers can integrate into applications.

Imagine an online hotel booking system.

A customer books a room.

The system could trigger:

Booking completed

↓

API

↓

SMS confirmation

↓

Customer receives message

The hotel does not necessarily need to build its own global SMS infrastructure.

4. Ecosystem Governance

This is one of the most important concepts in the slide.

If a company opens its APIs to thousands of partners, it needs rules.

Otherwise, the ecosystem can become unsafe or difficult to manage.

Ecosystem governance means deciding:

- Who can access the API?
- What data can they access?
- What can they do with the data?
- How is access authenticated?
- How is data protected?
- How much does API usage cost?
- What happens if an API is abused?
- What standards must partners follow?

API Authentication

Imagine an application wants to use a company's API.

The API provider needs to know:

"Who are you, and are you authorised to access this service?"

Common mechanisms include:

- API keys
- OAuth
- Access tokens
- Authentication credentials

Example

A student develops:

MyFoodApp

MyFoodApp wants to access a payment API.

The payment provider gives MyFoodApp credentials.

The application sends authorised requests.

The API verifies the request.

Then the API responds.

Simplified process

MyFoodApp

→ Authentication

→ API request

→ API verifies credentials

→ API processes request

→ Response

Data Compliance

Another governance issue is **data protection**.

Imagine a healthcare application sharing patient information through APIs.

The system cannot simply allow every partner to access everything.

Instead:

Partner A → only information required for its function

Partner B → different authorised information

Unauthorised application → access denied

This principle can be explained to students as:

Give users and applications only the access they actually need.

This is particularly important for sensitive information such as:

- Financial information
- Healthcare information
- Personal information
- Authentication information